

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Green Fields, Vaddeswaram, Guntur, Andhra Pradesh - 522302

Department of Computer Science and Engineering (CSE-3)

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Cloud-Native AI Model Deployment Platform

Submitted By

Vemuri Venkata Naga Sai Sri Harshad (2300031215)

Pinnamaneni Rohith Venkata Sai (2300031556)

Ventrapragada Jyothi Swaroop (2300032529)

Mullapudi Gowtham Chowdary (2300033079)

Under the Supervision of

MR. R RANJITH KUMAR (EMP. ID: 7026)

August 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Cloud-Native AI Model Deployment Platform. It serves as an agreement between the development team and supervising faculty, and is intended for use by students, the project supervisor, and all capstone evaluation stakeholders.

1.2 Scope

The Cloud-Native AI Model Deployment Platform (referred to as "the Platform") is a cloud-native web application that enables developers and data scientists to upload, manage, deploy, and monitor machine learning (ML) models in a production environment. It uses Docker, Kubernetes, microservices, CI/CD pipelines, and cloud infrastructure (AWS / Azure / GCP) to provide scalable, secure, and highly available ML model serving.

The Platform includes the following major subsystems:

- Model Registry -- upload, version, and manage ML models
- API Gateway -- authentication, request routing, and rate limiting
- Inference Service -- load models and serve predictions via REST APIs
- CI/CD Pipeline -- automated build, test, and deployment workflow
- Monitoring & Logging -- real-time metrics and centralized log management
- Edge Client -- Raspberry Pi integration for edge inference demonstration
- Web Dashboard -- React.js-based frontend for platform management

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
AI	Artificial Intelligence
ML	Machine Learning
MLOps	Machine Learning Operations - practices for deploying ML systems in production
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
CI/CD	Continuous Integration / Continuous Deployment
Docker	Platform for building and running containerized applications
Kubernetes (K8s)	Open-source container orchestration system
JWT	JSON Web Token - used for stateless authentication
RBAC	Role-Based Access Control
HPA	Horizontal Pod Autoscaler (Kubernetes)
AWS / Azure / GCP	Amazon Web Services / Microsoft Azure / Google Cloud Platform
ONNX	Open Neural Network Exchange - interoperable ML model format

Term / Acronym	Definition
FR / NFR	Functional Requirement / Non-Functional Requirement

1.4 References

- Project Proposal: Cloud-Native AI Model Deployment Platform (Project 12)
- Capstone Abstract - KL University CSE-3, August 2026
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Kubernetes Documentation: <https://kubernetes.io/docs/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- MLOps Industry Survey, 2023 - Gartner & O'Reilly Media

1.5 Document Overview

- Section 2 -- Overall Description: product context, stakeholder analysis, user types, constraints
- Section 3 -- Functional Requirements: feature-level requirements per module
- Section 4 -- Non-Functional Requirements: performance, security, scalability
- Section 5 -- System Constraints: hardware, software, and regulatory limits
- Section 6 -- External Interface Requirements
- Section 7 -- System Architecture Overview
- Section 8 -- Use Cases

2. Overall Description

2.1 Product Perspective

The Platform bridges the gap between ML model development and production deployment. Traditional approaches are manual, monolithic, and difficult to scale. The Platform replaces them with an automated, containerized, microservices-based pipeline integrated with cloud infrastructure. It operates as a cloud-hosted service accessed via a web browser, with an optional Raspberry Pi 4 edge client for edge computing scenarios.

2.2 Product Functions

- Model Upload & Versioning -- developers upload trained ML models; system stores and versions them
- Model Validation -- validates uploaded models for format compatibility before deployment
- Automated Deployment -- CI/CD pipelines containerize and deploy models to Kubernetes automatically
- Prediction Serving -- Inference Service exposes REST API endpoints for real-time predictions
- Secure Access -- JWT authentication and RBAC control who can upload, deploy, and invoke models
- Autoscaling -- Kubernetes HPA scales inference pods based on traffic demand
- Monitoring -- Prometheus + Grafana provide real-time metrics and dashboards
- Log Management -- centralized log collection enables administrators to diagnose failures
- Edge Inference Demo -- Raspberry Pi validates edge-to-cloud connectivity

2.3 Stakeholder Analysis

2.3.1 Background - The Problem We Identified

During our research phase, we identified a widely documented problem: the disconnect between training a machine learning model and deploying it into live production. According to the 2023 O'Reilly AI Adoption in the Enterprise report, over 85% of ML models developed by organizations never reach production. Of those that do, the average time from model completion to live deployment exceeds 90 days due to manual, fragmented, and error-prone processes. We mapped this problem across several real-world user groups, each facing a distinct version of the same challenge.

2.3.2 Identified Stakeholder Groups

Stakeholder 1 -- ML Engineers and Data Scientists

Attribute	Details
Who they are	Professionals who design, train, and evaluate ML models in Jupyter notebooks and Python scripts.

Attribute	Details
Core problem	After building a model, there is no standard path to production. Deployment requires DevOps skills most data scientists do not have, taking days to weeks per model.
Pain points	1. No versioning of deployed models. 2. No monitoring of prediction quality or service health. 3. Skill gap: forced to learn DevOps or wait for another team. 4. Slow iteration: every update restarts the manual process.
How we help	A single upload triggers automated containerization and deployment. Versioning, rollback, and monitoring are built in - no DevOps knowledge required.

Stakeholder 2 -- DevOps and Platform Engineers

Attribute	Details
Who they are	Engineers responsible for CI/CD pipelines, cloud infrastructure, and container clusters.
Core problem	Every ML model deployed is a bespoke infrastructure effort - custom Docker images, Kubernetes manifests, and CI scripts with no standardization across projects.
Pain points	1. No reusable ML deployment template. 2. Rollback is manual and risky. 3. Resource wastage due to no autoscaling. 4. Observability gaps - no standard metrics or logs per ML service.
How we help	Reusable Docker + Kubernetes + GitHub Actions pipeline that any model can plug into. Auto-rollback, HPA, and built-in Prometheus/Grafana remove per-project infrastructure effort.

Stakeholder 3 -- Software Development Teams (API Consumers)

Attribute	Details
Who they are	Backend and full-stack developers who need ML-powered features (fraud detection, recommendations, classification) in their applications.
Core problem	ML models exist as notebook files - not callable services. Developers wait weeks for a data science team to expose an API, or embed models directly, which creates fragile code.
Pain points	1. No stable prediction endpoint with guaranteed uptime. 2. No consistent API schema or documentation. 3. Model updates break client code without warning. 4. No authentication or rate limiting on model endpoints.
How we help	Versioned, authenticated REST endpoints for every deployed model with stable JSON contracts, JWT security, and rate limiting.

Stakeholder 4 -- IT Operations and System Administrators

Attribute	Details
Who they are	Admins responsible for health, security, and availability of production systems.

Attribute	Details
Core problem	ML inference workloads have unpredictable, spiky resource demands. Without autoscaling and monitoring, admins either over-provision (wasting money) or face outages.
Pain points	1. No visibility into model service health, latency, or error rates. 2. Model endpoints exposed without authentication or RBAC. 3. No centralized logging for failure diagnosis. 4. Manual scaling cannot respond to traffic spikes quickly enough.
How we help	Grafana dashboards and Prometheus alerts provide real-time visibility. Kubernetes HPA handles autoscaling. JWT + RBAC enforces access control. Structured logs enable fast diagnosis.

Stakeholder 5 -- Business Analysts and Product Managers

Attribute	Details
Who they are	Decision-makers who commission ML features and depend on predictions to drive business outcomes.
Core problem	Business stakeholders experience 90+ day delays between requesting an ML feature and seeing it live. Model improvements are rare because redeployment is expensive.
Pain points	1. Extremely long time-to-production for ML features. 2. No visibility into whether deployed models are performing as expected. 3. Redeployment of improved models requires the same lengthy process.
How we help	Automated CI/CD reduces deployment time from days to minutes. Model versioning enables rapid iteration. Monitoring dashboards track model health over time.

Stakeholder 6 -- Healthcare Organizations

Attribute	Details
Who they are	Hospitals, diagnostic labs, and health-tech startups deploying clinical AI for diagnostics or patient risk stratification.
Core problem	Clinical AI models require strict version control (for regulatory compliance), high availability, and audit logs (for accountability).
Pain points	No model version history for audit trails; no access control on sensitive model endpoints; downtime risk from manual deployments.
How we help	Model Registry maintains full version history. RBAC ensures only authorized personnel access clinical endpoints. Kubernetes provides 99%+ uptime with auto-healing pods.

Stakeholder 7 -- Financial Institutions

Attribute	Details
Who they are	Banks, NBFCs, and fintech companies deploying fraud detection, credit scoring, and AML models.
Core problem	Financial ML models need extremely low-latency responses, high throughput, and guaranteed availability - manual deployments cannot meet these SLAs.

Attribute	Details
Pain points	High request volumes overwhelm single-instance deployments; manual rollback during failures is too slow for real-time fraud detection; no audit logs for compliance.
How we help	HPA scales inference pods to handle transaction spikes. Auto-rollback restores the previous model within minutes. All prediction requests are logged for compliance.

Stakeholder 8 -- Small & Medium Enterprises and Startups

Attribute	Details
Who they are	Early-stage startups and SMEs with ML capabilities but no dedicated MLOps or DevOps team.
Core problem	Startups cannot afford the engineering hours for a custom deployment pipeline. Commercial ML serving platforms (AWS SageMaker, Google Vertex AI) are prohibitively expensive.
Pain points	High cost of managed ML platforms; lack of in-house DevOps expertise; inability to scale without major rework.
How we help	Platform uses only open-source tools on cloud free-tier resources, making production ML deployment accessible without specialized expertise or significant cost.

Stakeholder 9 -- Academic and Research Institutions

Attribute	Details
Who they are	University research groups and government labs that want to expose trained models for external use or collaborative research.
Core problem	Research models are shared as code or weight files with no deployment infrastructure. External researchers must set up the entire inference stack themselves.
Pain points	Models shared as GitHub repositories with no callable service; no versioning; no access control; no scalability if widely adopted.
How we help	Researchers register a model and share a stable, versioned API endpoint. API key authentication controls access. Model Registry preserves the full version history.

2.3.3 Stakeholder Summary

Stakeholder	Core Pain Point	Platform Feature Addressing It
ML Engineer / Data Scientist	No standard automated deployment path	Model Registry + CI/CD auto-deploy
DevOps / Platform Engineer	No reusable ML deployment template	Docker + K8s + GitHub Actions pipeline
Software Developer (API Consumer)	No stable, secure prediction endpoint	REST API Gateway + JWT + versioned endpoints
IT Operations / SysAdmin	No monitoring, manual scaling	Prometheus + Grafana + K8s HPA autoscaling
Business Analyst / PM	90+ day deployment delays	CI/CD reduces deploy time to minutes
Healthcare Organization	Need audit trails and access control	Model versioning + RBAC + audit logs
Financial Institution	Low-latency, high-throughput SLAs	K8s HPA + auto-rollback + request logs
SME / Startup	Too expensive / complex to deploy ML	Open-source stack on cloud free-tier
Academic / Research Institution	No shareable, versioned model API	Registry + stable endpoints + API keys

2.4 User Classes and Characteristics (Platform Roles)

User Role	Maps To Stakeholder	Privileges
Administrator	IT Operations / SysAdmin	Full access: user mgmt, RBAC, deployments, monitoring, logs
ML Engineer	ML Engineer / Data Scientist	Upload models, trigger deployments, view metrics, manage versions
Data Scientist	Data Scientist (read-heavy)	Browse registry, invoke inference API, view dashboards - no deploy
API Consumer	Software Developer / Business System	Authenticated inference API calls only - no dashboard access
Edge Client	IoT / Raspberry Pi device	API-key-based inference calls only - no human-facing interface

2.5 Operating Environment

- Cloud Platform: AWS, Microsoft Azure, or GCP (Free Tier)
- Container Runtime: Docker 24+
- Container Orchestration: Kubernetes 1.28+
- Backend Runtime: Java 17+ (Spring Boot), Python 3.10+ (FastAPI)
- Frontend: Node.js 18+ with React.js served via Nginx
- Database: PostgreSQL 15+ | CI/CD: GitHub Actions

- Monitoring: Prometheus 2.x + Grafana 10.x
- Edge Device: Raspberry Pi 4 Model B (4 GB RAM), Raspberry Pi OS
- Client Browser: Chrome, Firefox, or Edge (modern versions)

2.6 Design and Implementation Constraints

- System must use cloud-native architecture (containers + Kubernetes) - no monolithic deployment
- All inter-service communication must use REST APIs over HTTPS
- Cloud free-tier limits constrain compute and storage in the prototype phase
- ML models must be in supported formats: ONNX, TensorFlow SavedModel, or scikit-learn pickle
- Authentication must be stateless - JWT mandatory; no server-side session storage
- CI/CD pipeline must be version-controlled in GitHub and trigger automatically on push
- Raspberry Pi edge client must communicate over WiFi using HTTPS

2.7 Assumptions and Dependencies

- Cloud platform (AWS/Azure/GCP) free-tier accounts are available throughout the project
- Team has access to KL University laboratory laptops for development
- Trained ML models are provided externally for testing - model training is out of scope
- GitHub repositories are accessible by all four team members
- The Raspberry Pi 4 has stable WiFi connectivity to the internet
- PostgreSQL is hosted as a containerized instance within Kubernetes or as a managed cloud service

3. Functional Requirements

Requirements are identified with unique IDs in the format FR-<Module>-<Number>.

3.1 User Authentication and Authorization

- FR-AUTH-01** The system shall allow users to register with a valid email address and password.
- FR-AUTH-02** The system shall authenticate users using JWT tokens with a configurable expiry (default: 1 hour).
- FR-AUTH-03** The system shall enforce RBAC with roles: Administrator, ML Engineer, Data Scientist, API Consumer.
- FR-AUTH-04** The system shall reject unauthorized requests with HTTP 401 (Unauthorized) or HTTP 403 (Forbidden).
- FR-AUTH-05** An Administrator shall be able to create, update, deactivate, and delete user accounts.
- FR-AUTH-06** The system shall support API key generation for external API consumers as an alternative to JWT.

3.2 Model Registry

- FR-REG-01** An authenticated ML Engineer shall be able to upload a trained ML model file (ONNX, TensorFlow SavedModel, or scikit-learn pickle).
- FR-REG-02** The system shall store model metadata: name, version, description, upload timestamp, uploader identity, file size, and framework.
- FR-REG-03** The system shall assign a unique version identifier to each uploaded model automatically.
- FR-REG-04** The system shall validate the uploaded model file format and reject invalid or corrupted files with a descriptive error.
- FR-REG-05** An ML Engineer shall be able to list all models and their versions in the registry.
- FR-REG-06** An ML Engineer shall be able to mark a specific model version as "Active" to trigger deployment.
- FR-REG-07** An Administrator shall be able to delete a model version; active/deployed versions require explicit confirmation.
- FR-REG-08** The registry shall store model files in persistent cloud object storage separate from the database.

3.3 Model Deployment (CI/CD Pipeline)

- FR-CICD-01** The CI/CD pipeline shall automatically trigger on a model "Active" event or a code push to the main GitHub branch.
- FR-CICD-02** The pipeline shall build a Docker image containing the Inference Service and the active model.
- FR-CICD-03** The pipeline shall run automated validation tests on the built Docker image before deployment.
- FR-CICD-04** The pipeline shall push the validated Docker image to a container registry (Docker

Hub or cloud registry).

- FR-CICD-05** The pipeline shall deploy the new image to Kubernetes using rolling update with zero downtime.
- FR-CICD-06** The pipeline shall notify the platform dashboard with deployment status on completion.
- FR-CICD-07** An Administrator shall be able to view deployment history and pipeline logs from the web dashboard.
- FR-CICD-08** On deployment failure, the system shall automatically roll back to the previously stable version.

3.4 Inference Service

- FR-INF-01** The Inference Service shall expose a REST endpoint (POST /api/v1/predict) to accept prediction requests.
- FR-INF-02** The endpoint shall accept input data in JSON format and return prediction results in JSON format.
- FR-INF-03** The Inference Service shall load the active model at startup and serve predictions without restarting per request.
- FR-INF-04** The system shall return HTTP 200 with prediction results for valid requests.
- FR-INF-05** The system shall return descriptive error responses (HTTP 400/422) for malformed or invalid input.
- FR-INF-06** The Inference Service shall support concurrent prediction requests using asynchronous processing.
- FR-INF-07** Kubernetes HPA shall scale Inference Service pods when CPU utilization exceeds 70%.
- FR-INF-08** The Inference Service shall record each prediction request in the database for auditing.

3.5 API Gateway

- FR-GW-01** All external requests to backend microservices shall pass through the API Gateway.
- FR-GW-02** The API Gateway shall authenticate incoming requests by validating JWT tokens or API keys before routing.
- FR-GW-03** The API Gateway shall route requests to the correct downstream microservice based on URL path.
- FR-GW-04** The API Gateway shall enforce rate limiting: maximum 100 requests per minute per authenticated user.
- FR-GW-05** The API Gateway shall return HTTP 429 (Too Many Requests) when rate limits are exceeded.
- FR-GW-06** The API Gateway shall log all requests: timestamp, user ID, endpoint, HTTP method, and response status.

3.6 Monitoring and Logging

- FR-MON-01** Prometheus shall collect metrics including CPU usage, memory, pod count, request rate, error rate, and inference latency.
- FR-MON-02** Grafana shall display real-time dashboards for all Prometheus metrics.
- FR-MON-03** The system shall send an alert when CPU usage exceeds 80% for more than 5

consecutive minutes.

FR-MON-04 The system shall send an alert when the prediction error rate exceeds 5%.

FR-MON-05 All microservices shall output structured JSON logs to a centralized logging system.

FR-MON-06 An Administrator shall be able to query and filter logs by service, time range, and severity level.

FR-MON-07 The system shall retain logs for a minimum of 30 days.

3.7 Edge Computing Client (Raspberry Pi)

FR-EDGE-01 The Raspberry Pi client shall authenticate with the platform using a pre-configured API key.

FR-EDGE-02 The edge client shall collect sensor or image data and submit it to the Inference Service REST API.

FR-EDGE-03 The edge client shall display the received prediction result locally (terminal output or LED indicator).

FR-EDGE-04 The edge client shall retry failed requests up to 3 times before logging an error.

FR-EDGE-05 All communication between the Raspberry Pi and the cloud platform shall use HTTPS.

3.8 Web Dashboard (Frontend)

FR-UI-01 The web dashboard shall provide a login page with email, password fields, and a sign-in button.

FR-UI-02 After login, the dashboard shall display a summary: number of models, active deployments, and system health.

FR-UI-03 The dashboard shall include a Model Registry page for uploading models and managing versions.

FR-UI-04 The dashboard shall include a Deployments page showing pipeline history and rollback controls.

FR-UI-05 The dashboard shall embed Grafana monitoring dashboards for real-time metrics.

FR-UI-06 The dashboard shall include an API Testing panel for submitting sample predictions.

FR-UI-07 The dashboard shall be responsive and usable on desktop browsers (minimum 1024px viewport).

FR-UI-08 An Administrator shall access a User Management page to manage accounts and roles.

4. Non-Functional Requirements

4.1 Performance

- NFR-PERF-01** The Inference Service shall respond within 500 ms under normal load (up to 50 concurrent requests).
- NFR-PERF-02** Web dashboard pages shall load within 3 seconds on a standard broadband connection.
- NFR-PERF-03** The CI/CD pipeline shall complete a full build-test-deploy cycle within 10 minutes.
- NFR-PERF-04** The system shall support a minimum of 100 concurrent prediction requests without degradation.

4.2 Scalability

- NFR-SCALE-01** The Inference Service shall scale horizontally from 1 to a minimum of 5 pod instances based on traffic.
- NFR-SCALE-02** The Model Registry and database shall support at least 50 model versions without performance impact.
- NFR-SCALE-03** The platform architecture shall support adding new microservices without modifying existing ones.

4.3 Availability and Reliability

- NFR-AVAIL-01** The platform shall achieve a minimum uptime of 99% during the demonstration period.
- NFR-AVAIL-02** Kubernetes liveness and readiness probes shall restart failed pods within 30 seconds.
- NFR-AVAIL-03** The system shall use rolling deployment updates to ensure zero-downtime deployments.
- NFR-AVAIL-04** On a failed deployment, the system shall roll back to the last stable version within 5 minutes.
- NFR-AVAIL-05** The PostgreSQL database shall have automated daily backups retained for 7 days.

4.4 Security

- NFR-SEC-01** All API communication shall use HTTPS/TLS 1.2+; plain HTTP connections shall be rejected.
- NFR-SEC-02** User passwords shall be stored as salted bcrypt hashes; plain-text passwords shall never be stored.
- NFR-SEC-03** JWT tokens shall include expiry claims and be invalidated on logout via a token denylist.
- NFR-SEC-04** RBAC shall prevent lower-privileged users from accessing administrator endpoints.
- NFR-SEC-05** All uploaded model files shall be scanned for malicious content before storage.
- NFR-SEC-06** API keys shall be stored as SHA-256 hashes; raw keys are displayed only once at

generation.

NFR-SEC-07 Accounts shall be blocked for 15 minutes after 5 consecutive failed login attempts.

4.5 Maintainability

NFR-MAINT-01 Each microservice shall be independently deployable without changes to other services.

NFR-MAINT-02 Source code shall be maintained in Git with meaningful commit messages and PR reviews.

NFR-MAINT-03 Breaking API changes shall require a new API version (e.g., /api/v2/).

NFR-MAINT-04 Infrastructure configuration shall be managed as code using Kubernetes YAML manifests in version control.

4.6 Portability

NFR-PORT-01 All microservices shall be containerized and deployable on any Kubernetes-compatible cloud provider.

NFR-PORT-02 The platform shall not use proprietary cloud-vendor APIs that prevent migration between providers.

NFR-PORT-03 Docker Compose configuration shall be provided for local development without a full Kubernetes cluster.

5. System Constraints

5.1 Hardware Constraints

Component	Qty	Purpose	Cost (INR)
Raspberry Pi 4 (4 GB RAM)	1	Edge client for inference testing	4,000
MicroSD Card 64 GB	1	Raspberry Pi OS storage	600
Laptop / Dev System	1	Development and local testing (Lab Resource)	0
External SSD	1	Additional storage for datasets / models	3,500
WiFi Router / Network	1	Network connectivity for all devices	1,000
Cloud Platform (AWS/Azure/GCP)	1	K8s, database, and CI/CD hosting	0 (Free Tier)
Cooling Fan	1	Raspberry Pi thermal management	400
Cables & Accessories	1 set	Power, data, and network cables	800
TOTAL			9,000 - 15,000

Note: High-cost hardware (enterprise server Rs.50,000+, GPU cluster Rs.70,000+, data center hardware Rs.1,00,000+) has been excluded. Equivalent capabilities are provided by cloud infrastructure at zero cost within free-tier limits.

5.2 Software Constraints

- All services must be containerized using Docker - no bare-metal or VM-only deployments
- Backend restricted to Spring Boot (Java) and FastAPI (Python)
- Frontend restricted to React.js | Database must be PostgreSQL
- CI/CD must use GitHub Actions; Jenkins or GitLab CI are out of scope
- Monitoring must use the Prometheus + Grafana stack
- ML model training is explicitly out of scope - only deployment and inference are in scope

6. External Interface Requirements

6.1 User Interfaces

- Sidebar: Dashboard Overview, Model Registry, Deployments, Monitoring, API Testing, User Management
- Responsive layout for desktop browsers (minimum 1024px width)
- Color-coded status indicators: green (healthy), yellow (warning), red (error/failed)
- Drag-and-drop file upload for model submission
- Auto-refresh monitoring dashboards every 30 seconds
- Login page with email and password fields; no client-side password storage

6.2 Hardware Interfaces

- Raspberry Pi 4 connects to the cloud Inference Service via WiFi over HTTPS

- Python-based client script reads sensor/image input and calls the prediction REST API
- No direct hardware interface beyond standard TCP/IP networking

6.3 Software Interfaces

Interface	Technology	Purpose
Model Registry API	Spring Boot REST	CRUD operations on models and versions
Inference API	FastAPI (Python)	Accept prediction requests and return results
API Gateway	Spring Cloud Gateway	Route, authenticate, and rate-limit requests
Database	PostgreSQL via JDBC / SQLAlchemy	Persistent storage for metadata and logs
Container Registry	Docker Hub or AWS ECR	Store and version Docker images
CI/CD	GitHub Actions YAML	Automated build, test, and deploy pipeline
Metrics	Prometheus /metrics endpoint	Collect application and infrastructure metrics
Visualization	Grafana (Prometheus data source)	Dashboards and alerting
Model File Storage	AWS S3 / Azure Blob / GCP GCS	Persist uploaded ML model files

6.4 Communication Interfaces

- All client-to-server and service-to-service communication shall use HTTPS (TLS 1.2+) on port 443
- Internal Kubernetes communication uses HTTP within the cluster (protected by network policies)
- Authentication tokens passed in HTTP Authorization header: Authorization: Bearer <JWT>
- API requests and responses use JSON (Content-Type: application/json)
- Prometheus metrics on HTTP port 8080 - accessible only within the cluster
- Real-time log streaming may use WebSockets (WSS) for dashboard live updates

7. System Architecture Overview

7.1 Architecture Style

The Platform follows a Microservices Architecture where each functional component is an independent, loosely coupled service deployed as a Docker container within a Kubernetes cluster. Services communicate via REST APIs routed through the API Gateway.

7.2 Core Microservices

Microservice	Technology	Responsibility
Auth Service	Spring Boot (Java)	User registration, login, JWT issuance, RBAC enforcement
Model Registry Service	Spring Boot (Java)	Model upload, versioning, metadata management, file storage
Inference Service	FastAPI (Python)	Load active ML model and serve predictions via REST
API Gateway	Spring Cloud Gateway	Entry point: authentication, routing, rate limiting
Monitoring Stack	Prometheus + Grafana	Metrics collection, alerting, and visualization
Frontend	React.js via Nginx	Web dashboard for all user interactions
Database	PostgreSQL	Users, model metadata, audit logs

7.3 Prediction Request Data Flow

- Step 1: User / Edge Client sends POST /api/v1/predict to the Ingress endpoint.
- Step 2: API Gateway validates the JWT / API key; rejects unauthorized requests with HTTP 401.
- Step 3: API Gateway routes the request to the Inference Service.
- Step 4: Inference Service runs the input through the active ML model and generates predictions.
- Step 5: Prediction result returned as JSON; response travels back through the Gateway to the client.
- Step 6: Request metadata logged to PostgreSQL; metrics emitted to Prometheus.

8. Use Cases

UC-01: Upload and Register ML Model

Field	Detail
Use Case ID	UC-01
Name	Upload and Register ML Model
Actor	ML Engineer
Precondition	Actor is authenticated with ML Engineer role
Main Flow	1. Actor navigates to Model Registry page. 2. Actor provides model name, version description, and selects model file. 3. Actor clicks Upload. 4. System validates file format and integrity. 5. System stores model file and creates registry entry. 6. System displays success message with model ID and version.
Alternate Flow	Step 4: If file is invalid, system returns error message and prompts re-upload.
Postcondition	Model listed in registry with status "Uploaded".
Related Requirements	FR-REG-01 through FR-REG-04

UC-02: Deploy Model to Production

Field	Detail
Use Case ID	UC-02
Name	Deploy Model to Production
Actor	ML Engineer
Precondition	A model is registered with status "Uploaded"
Main Flow	1. Actor selects a model version and clicks "Set Active". 2. CI/CD pipeline triggers automatically. 3. Pipeline builds Docker image with the model. 4. Pipeline runs tests and pushes image to registry. 5. Kubernetes performs rolling update. 6. Dashboard shows "Deployment Successful".
Alternate Flow	Step 4: If tests fail, pipeline stops; auto-rollback is initiated; dashboard shows "Deployment Failed".
Postcondition	New model version serving predictions; previous version decommissioned.
Related Requirements	FR-CICD-01 through FR-CICD-08

UC-03: Submit a Prediction Request

Field	Detail
Use Case ID	UC-03
Name	Submit a Prediction Request

Field	Detail
Actor	API Consumer / Edge Client
Precondition	A model is actively deployed; actor has valid API key or JWT
Main Flow	1. Client sends POST /api/v1/predict with JSON payload and Authorization header. 2. API Gateway validates token. 3. Request routed to Inference Service. 4. Inference Service returns JSON prediction. 5. Client receives HTTP 200 with result.
Alternate Flow	Invalid token -> HTTP 401. Malformed input -> HTTP 422. Rate limit exceeded -> HTTP 429.
Postcondition	Prediction result returned; request logged in the database.
Related Requirements	FR-INF-01 to FR-INF-08, FR-GW-01 to FR-GW-06

UC-04: Monitor System Health

Field	Detail
Use Case ID	UC-04
Name	Monitor System Health
Actor	Administrator
Precondition	Administrator is logged in; Prometheus and Grafana are running
Main Flow	1. Administrator navigates to the Monitoring section. 2. Grafana dashboards display CPU/memory, request rate, error rate, pod count, inference latency. 3. Administrator views real-time and historical data. 4. Alert notification appears if a threshold is breached.
Alternate Flow	If Prometheus is unreachable, Grafana displays "No Data" warning.
Postcondition	Administrator is informed of system health and can take corrective action.
Related Requirements	FR-MON-01 through FR-MON-07

9. Appendix

9.1 Requirement Traceability Matrix

Req. ID	Description	Section	Priority
FR-AUTH-01-06	User registration, JWT auth, RBAC, API keys	3.1	High
FR-REG-01-08	Model upload, versioning, metadata, validation	3.2	High
FR-CICD-01-08	CI/CD pipeline, Docker build, K8s deploy, rollback	3.3	High
FR-INF-01-08	Prediction REST API, concurrent serving, HPA, audit log	3.4	High
FR-GW-01-06	API Gateway routing, auth, rate limiting, logging	3.5	High
FR-MON-01-07	Prometheus metrics, Grafana dashboards, alerts, logs	3.6	Medium
FR-EDGE-01-05	Raspberry Pi edge client, HTTPS, retry logic	3.7	Medium
FR-UI-01-08	React.js dashboard: login, registry, deployments, metrics	3.8	High
NFR-PERF-01-04	Inference <500ms, pipeline <10min, 100 concurrent users	4.1	High
NFR-SCALE-01-03	HPA 1-5 pods, 50+ model versions, extensible architecture	4.2	Medium
NFR-AVAIL-01-05	99% uptime, auto-restart, zero-downtime, auto-rollback	4.3	High
NFR-SEC-01-07	HTTPS/TLS, bcrypt, JWT denylist, RBAC, file scanning	4.4	High
NFR-MAINT-01-04	Independent deployability, Git versioning, API versioning	4.5	Medium
NFR-PORT-01-03	Multi-cloud portability, Docker Compose for local dev	4.6	Low

9.2 Document Revision History

Version	Date	Author	Changes
1.0	August 2026	Pinnamaneni Rohith Venkata Sai	Initial SRS created for capstone presentation

End of Software Requirements Specification

Cloud-Native AI Model Deployment Platform | KL University | August 2026